

Edge Detection using Sobel Filter in FPGA

Report submitted as part of the project for
Digital System Design course under
the Bachelor of Technology degree in

Electronics and Communication Engineering

by

Utkarsh Kumar - 23UEC637
Vaibhav Garg - 23UEC638
Vibha Gupta - 23UEC642

Course Instructor

Dr. Kusum Lata



**Department of Electronics and Communication
Engineering**

The LNM Institute of Information Technology, Jaipur

April 27, 2026

Contents

1	Objective	3
2	Tools and Resources Used	3
3	System Design / Architecture	3
3.1	Block Diagram	3
3.2	Block Diagram	4
3.3	System Description	5
3.4	Design Details	5
3.5	Resource Utilization Plan	5
4	Implementation	6
4.1	Design Implementation on Nexys 4 DDR	6
4.2	Working of the System	6
4.3	Configuration Steps	6
4.4	Clock Constraints	7
4.5	Pin Assignments	7
5	Simulation and Testing	7
5.1	Functional Simulation	7
5.2	On-Board Testing	7
5.3	Observed vs Expected Behavior	7
6	Results	8
6.1	Output: Edge Detected Image - 1	8
6.2	Output: Edge Detected Image - 2	9
6.3	Observation:	10
6.4	Schematic	11
6.5	Simulation	11
6.6	Project Summary	13
7	Challenges Faced and Mitigation	14
8	Future Improvements	15
9	Conclusion	15
10	References	16
11	Appendix	16
11.1	Top Module	16
11.2	Line Buffer Module	18
11.3	Sobel Module	20
11.4	VGA Controller	20
11.5	Python Code	21

11.6 Constraint File	22
--------------------------------	----

Project - "Edge Detection using Sobel Filter in FPGA"

1 Objective

In this project, the goal is to design and build an edge detector for a grayscale image using the Sobel operator on an FPGA. An image is loaded into on-chip memory, and edges are detected based on changes in intensity levels between adjacent pixels.

The project will cover the following aspects:

- Implementing a hardware-efficient Sobel edge detection algorithm
- Processing image data in a streaming manner without storing the full frame
- Designing a pipelined architecture for high-speed operation
- Displaying the processed output on a VGA monitor in real time

This work demonstrates how FPGA-based systems can be used for fast and parallel image processing tasks.

2 Tools and Resources Used

- **Development Board:** Nexys 4 DDR (Artix-7 FPGA)
- **Design Software:** Xilinx Vivado 2023.2
- **Languages Used:** Verilog HDL, Python
- **Additional Tools:** OpenCV, NumPy, Vivado Simulator
- **IP Cores Used:** Block Memory Generator (BRAM IP)

3 System Design / Architecture

3.1 Block Diagram

The system consists of the following major modules:

- Block RAM (BRAM) for image storage
- Line Buffer for 3×3 window generation
- Sobel Processing Unit
- VGA Controller

3.2 Block Diagram

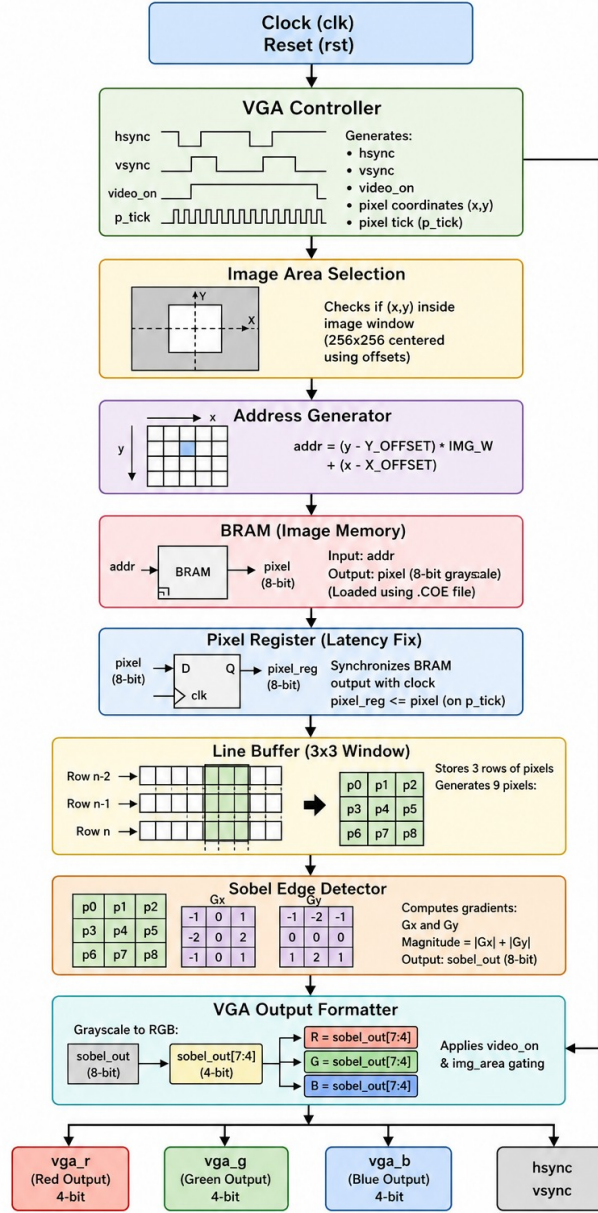


Figure 1: Block Diagram of the Proposed System

The system consists of BRAM for image storage, followed by a line buffer that generates a 3×3 pixel window. The Sobel processing unit computes edge magnitude, and the VGA controller displays the output on the screen.

3.3 System Description

The designed system performs real-time edge detection using the Sobel operator on an FPGA. The input is a 256×256 grayscale image stored in BRAM. The image is processed in a streaming manner, where one pixel is processed per clock cycle after an initial latency.

Each pixel is passed through a line buffer that generates a 3×3 neighborhood window required for convolution. The Sobel module computes horizontal and vertical gradients using this window and generates the edge magnitude.

The processed pixel values are then sent to a VGA controller, which displays the edge-detected image on a 640×480 screen using raster scanning.

3.4 Design Details

- **Image Representation:** 8-bit grayscale (0–255)
- **Memory Organization:** Row-major storage in BRAM
- **Address Calculation:**

$$\text{Address} = (\text{row} \times \text{image width}) + \text{column}$$

- **Sobel Equations:**

$$Gx = (p2 + 2p5 + p8) - (p0 + 2p3 + p6)$$

$$Gy = (p6 + 2p7 + p8) - (p0 + 2p1 + p2)$$

$$\text{Edge} = |Gx| + |Gy|$$

3.5 Resource Utilization Plan

The design uses:

- BRAM for image storage
- Registers for pipeline stages
- Combinational logic for Sobel computation

Pipeline architecture ensures high throughput with one pixel processed per clock cycle after initialization.

4 Implementation

4.1 Design Implementation on Nexys 4 DDR

The system is implemented on the Nexys 4 DDR FPGA board using Verilog HDL. The architecture integrates VGA timing, memory access, line buffering, and Sobel processing into a synchronized pipeline.

4.2 Working of the System

1. On reset, all modules including VGA controller, line buffer, and Sobel unit are initialized.
2. The input image is preprocessed using Python (grayscale conversion and resizing to 256×256) and stored in BRAM using a COE file.
3. The VGA controller generates pixel coordinates (x, y), synchronization signals (hsync, vsync), and a pixel tick signal.
4. Based on coordinates, pixel data is fetched from BRAM using computed addresses.
5. Due to BRAM latency, pixel data is registered before processing.
6. The line buffer stores incoming pixels and generates a 3×3 window once sufficient data is available.
7. The Sobel module computes gradients (Gx, Gy) and outputs edge magnitude.
8. The processed pixel is mapped to VGA RGB channels.
9. Pixels outside the image region are set to zero (black background).
10. After pipeline filling, the system outputs one processed pixel per clock cycle.

4.3 Configuration Steps

1. Create a new Vivado project targeting Nexys 4 DDR.
2. Add Verilog source files.
3. Generate BRAM IP and load COE file.
4. Add constraint (.xdc) file.
5. Run synthesis and implementation.
6. Generate bitstream.
7. Program FPGA using Hardware Manager.

4.4 Clock Constraints

```
create_clock -period 10.00 [get_ports clk]
```

4.5 Pin Assignments

- clk → System clock (100 MHz)
- rst → Reset button
- VGA signals → $vga_r, vga_g, vga_b, hsync, vsync$

5 Simulation and Testing

5.1 Functional Simulation

Functional simulation was performed using Vivado Simulator. The waveform verifies:

- Proper VGA synchronization signals (hsync, vsync)
- Correct pixel coordinate generation
- BRAM read operation with latency handling
- Line buffer generating valid 3×3 window
- Sobel output generation after valid input

5.2 On-Board Testing

1. Program the FPGA with generated bitstream
2. Connect VGA monitor
3. Reset the system
4. Observe output image on screen

5.3 Observed vs Expected Behavior

- Edge detection output matched expected results
- Smooth display without flickering
- Correct synchronization of VGA signals
- Continuous pixel processing after pipeline delay

6 Results

The design implementation proved successful when the design was successfully synthesized and tested on the FPGA board. This system can successfully process the input image present in the BRAM and output the edge-detected image on the VGA screen. These results show that all of the modules have been correctly implemented.

6.1 Output: Edge Detected Image - 1



Figure 2: First Input Image



Figure 3: Output of first input image

6.2 Output: Edge Detected Image - 2



Figure 4: Second Input Image



Figure 5: Output of second input image

6.3 Observation:

The result obtained after the images pass through the Sobel filter is displayed on the VGA display. Sharp variations in intensity are shown in white, while flat areas are represented in black.

The boundaries of the objects are well defined, proving that gradient calculation was properly executed. Horizontal and vertical edges are detected perfectly. There is no flickering in the results, which implies that timing synchronization has been achieved.

6.4 Schematic

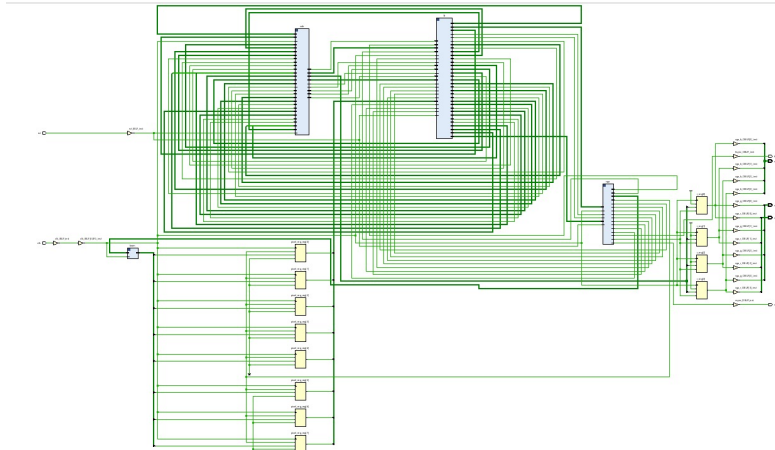
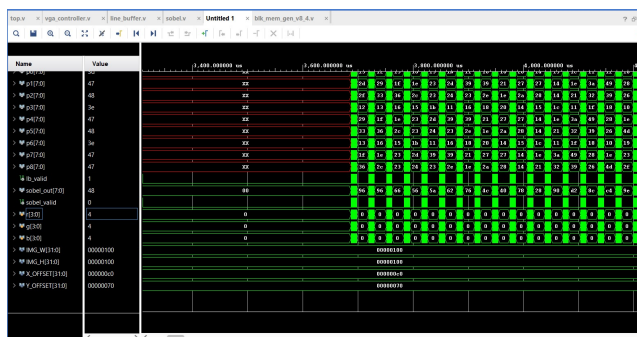
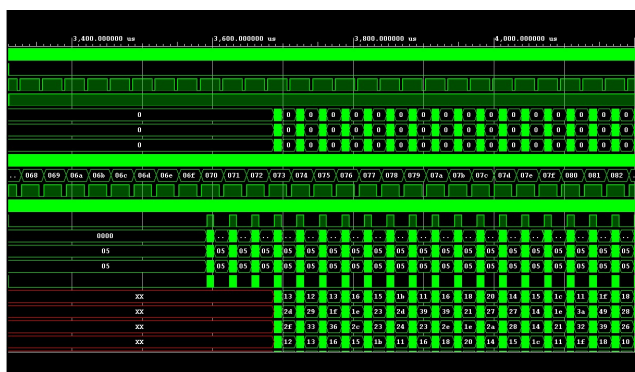


Figure 6: Schematic

6.5 Simulation



Figure 7: When R, G & B do not get Data



The waveforms depict the entire process of implementing the VGA-based image processing system using FPGA technology. The VGA controller outputs horizontal and vertical synchronization pulses ('hsync', 'vsync') in addition to the position of pixels ('x', 'y') and clock pulse for each pixel ('p_tick'). The output also includes the 'video_on' signal that specifies the area of the screen where valid pixel information exists. On the basis of the current position ('x', 'y') coordinates, the corresponding address is determined to extract pixel information from BRAM, which holds the input image. Because of the BRAM delay, the pixel information is latched ('pixel_reg').

The validity of the pixels used for Sobel filtering is determined by the ‘pixel_valid’ signal. In its turn, the initial value of the ‘lb’ output signal from the line buffer will be undefined (‘XX’), since the buffer requires accumulation of at least two lines to create the valid 3×3 window. Then, having enough data, the line buffer provides nine values of pixels (‘p0’ through ‘p8’). As soon as ‘lb_valid’ goes high, the window will become valid and go to the Sobel filter block. It calculates both horizontal and vertical components of gradients (‘Gx’ and ‘Gy’, correspondingly). Also, an output (‘sobel_out = −Gx − + −Gy−’)

indicating the magnitude of edges can be provided.

Lastly, the values of the processed pixels are assigned to the VGA RGB signals. From the waveform generated, the functionality of VGA timing control, memory accesses, generation of the line buffer window, and Sobel operator processing are all shown. The waveform also brings out the effect of the pipeline delay from the input to output sections.

6.6 Project Summary

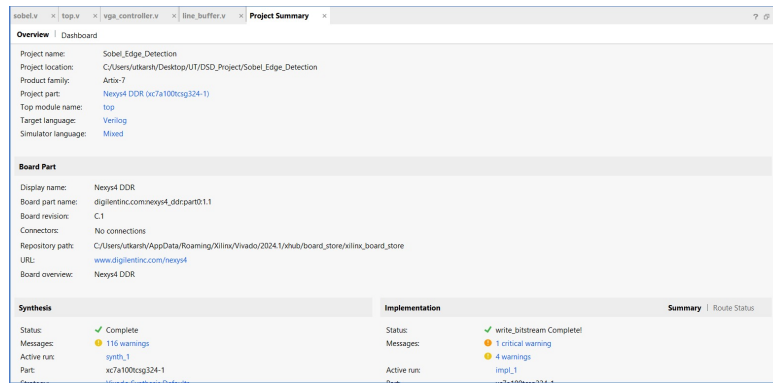


Figure 10: Project summary screenshot-1

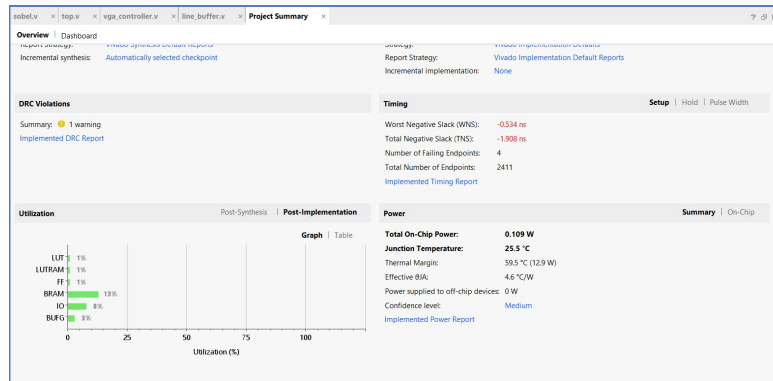


Figure 11: Project summary screenshot-2

Utilization				Post-Synthesis	Post-Implementation
				Graph	Table
Resource	Utilization	Available	Utilization %		
LUT	452	63400	0.71		
LUTRAM	224	19000	1.18		
FF	104	126800	0.08		
BRAM	17	135	12.59		
IO	16	210	7.62		
BUFG	1	32	3.13		

Figure 12: Resource utilization report

7 Challenges Faced and Mitigation

- Line Buffer Initialization Delay**
Initially, the Sobel module produced undefined outputs because the 3×3 window was not fully populated.
Mitigation: A valid signal (`lb_valid`) was used to ensure that processing begins only after sufficient rows and columns are available.
- BRAM Read Latency**
Direct use of BRAM output caused misalignment between pixel data and processing stages due to memory delay.
Mitigation: A register stage (`pixel_reg`) was introduced to synchronize pixel data with downstream modules.
- VGA Timing Synchronization**
Incorrect synchronization between pixel data and VGA signals resulted in distorted or shifted images.
Mitigation: Pixel processing was aligned with the `p_tick` signal generated by clock division to match VGA timing requirements.
- Pipeline Delay Handling**
The pipeline architecture introduced delays between input and output, making debugging difficult.
Mitigation: Simulation waveforms were analyzed to understand timing delays and ensure correct data flow across stages.
- Edge Magnitude Overflow**
The Sobel computation sometimes produced values exceeding the 8-bit range.
Mitigation: Output values were clipped to 255 using saturation logic.
- Display Region Alignment**
The processed image was not correctly centered on the VGA display initially.

Mitigation: X and Y offsets were adjusted to properly position the image within the visible area.

8 Future Improvements

This project implements Sobel edge detection on an FPGA. Future improvements could include:

- Introduction of pipelining delay to align pixels properly
- Extension to RGB images and several other image processing modes
- Integration of advanced algorithms for edge detection, like Canny method
- Implementation of real-time video acquisition with camera modules, such as OV7670
- Optimization of the design by implementing parallel/AXI-stream design architecture
- Ability to handle high-resolution images
- Dynamic thresholding for edge detection
- Evolution into an embedded vision system application

9 Conclusion

In this project, the edge detection system design and implementation based on FPGA using the Sobel edge detector has been accomplished. The image is first taken in grayscale format and kept in BRAM for further operations where edge detection takes place through the sliding window technique.

It can be noted that this design is successful in the application of the basic concepts of digital design like synchronous processing, streaming data flow, line buffering. The addition of the VGA controller has made sure that the results are properly displayed on the screen.

The simulation and the hardware results verify that all modules work correctly. The system can process pixels effectively and produce the output edges with reliable display results. The pipeline technique allows continuous processing with high throughput.

The whole project demonstrates that FPGA devices can perform complex computation operations on the images. It offers a good base for developing future hardware-based vision systems.

GitHub Link:

<https://github.com/utkarsh478/Hardware-Implementation-of-Sobel-Edge-Detection-on-FPGA>

10 References

- Xilinx Vivado Design Suite User Guide
- Xilinx Block Memory Generator (BRAM) IP Documentation
- Nexys 4 DDR FPGA Board Reference Manual
- Gonzalez and Woods, *Digital Image Processing*, Pearson
- OpenCV Documentation: <https://docs.opencv.org/>
- Sobel Operator Theory: https://en.wikipedia.org/wiki/Sobel_operator
- GitHub Repository: <https://github.com/utkarsh478/Hardware-Implementation-of-Sobel-Edge-D>

11 Appendix

11.1 Top Module

```
module top(  
    input clk,  
    input rst,  
    output hsync,  
    output vsync,  
    output [3:0] vga_r,  
    output [3:0] vga_g,  
    output [3:0] vga_b  
);  
  
parameter IMG_W = 256;  
parameter IMG_H = 256;  
parameter X_OFFSET = 192;  
parameter Y_OFFSET = 112;  
  
wire [9:0] x,y;  
wire video_on;  
wire p_tick;  
  
// VGA  
vga_controller vga(  
    .clk(clk),  
    .rst(rst),  
    .hsync(hsync),  
    .vsync(vsync),  
    .video_on(video_on),  
    .x(x),  
    .y(y),
```

```

        .p_tick(p_tick)
    );

    // Image area
    wire img_area;
    assign img_area = (x >= X_OFFSET && x < X_OFFSET + IMG_W &&
        y >= Y_OFFSET && y < Y_OFFSET + IMG_H);

    // Address
    wire [15:0] addr;
    assign addr = img_area ? ((y - Y_OFFSET)*IMG_W + (x - X_OFFSET)) : 0;

    // BRAM
    wire [7:0] pixel;
    blk_mem_gen_0 bram(
        .clka(clk),
        .addra(addr),
        .douta(pixel)
    );

    // latency fix
    reg [7:0] pixel_reg;
    always @(posedge clk) begin
        if (p_tick)
            pixel_reg <= pixel;
    end

    // Pixel valid (SYNCED)
    wire pixel_valid;
    assign pixel_valid = img_area && p_tick;

    // Line buffer
    wire [7:0] p0,p1,p2,p3,p4,p5,p6,p7,p8;
    wire lb_valid;

    line_buffer lb(
        .clk(clk),
        .rst(rst),
        .pixel_in(pixel_reg),
        .pixel_valid(pixel_valid),
        .p0(p0),.p1(p1),.p2(p2),
        .p3(p3),.p4(p4),.p5(p5),
        .p6(p6),.p7(p7),.p8(p8),
        .valid_out(lb_valid)
    );

```

```

// Sobel
wire [7:0] sobel_out;
wire sobel_valid;

sobel sob(
    .clk(clk),
    .rst(rst),
    .p0(p0), .p1(p1), .p2(p2),
    .p3(p3), .p4(p4), .p5(p5),
    .p6(p6), .p7(p7), .p8(p8),
    .valid_in(lb_valid),
    .pixel_out(sobel_out),
    .valid_out(sobel_valid)
);

// VGA output
reg [3:0] r,g,b;

always @(posedge clk) begin
    if (video_on && img_area) begin
        r <= sobel_out[7:4];
        g <= sobel_out[7:4];
        b <= sobel_out[7:4];
    end else begin
        r <= 0;
        g <= 0;
        b <= 0;
    end
end

assign vga_r = r;
assign vga_g = g;
assign vga_b = b;

endmodule

```

11.2 Line Buffer Module

```

module line_buffer(
    input clk,
    input rst,
    input [7:0] pixel_in,
    input pixel_valid,
    output reg [7:0] p0,p1,p2,p3,p4,p5,p6,p7,p8,
    output reg valid_out

```

```

);

parameter WIDTH = 256;

reg [7:0] line0 [0:WIDTH-1];
reg [7:0] line1 [0:WIDTH-1];
reg [7:0] line2 [0:WIDTH-1];

reg [7:0] col;
reg [1:0] row_count;
reg filled;

always @(posedge clk) begin
    if (rst) begin
        col <= 0;
        row_count <= 0;
        filled <= 0;
        valid_out <= 0;
    end
    else if (pixel_valid) begin

        line2[col] <= line1[col];
        line1[col] <= line0[col];
        line0[col] <= pixel_in;

        if (col == WIDTH-1) begin
            col <= 0;
            if (row_count < 2)
                row_count <= row_count + 1;
            else
                filled <= 1;
        end else col <= col + 1;

        if (filled && col > 0 && col < WIDTH-1) begin
            p0 <= line2[col-1]; p1 <= line2[col]; p2 <= line2[col+1];
            p3 <= line1[col-1]; p4 <= line1[col]; p5 <= line1[col+1];
            p6 <= line0[col-1]; p7 <= line0[col]; p8 <= line0[col+1];
            valid_out <= 1;
        end else valid_out <= 0;
    end
    else valid_out <= 0;
end

endmodule

```

11.3 Sobel Module

```
module sobel(
    input clk,
    input rst,
    input [7:0] p0,p1,p2,p3,p4,p5,p6,p7,p8,
    input valid_in,
    output reg [7:0] pixel_out,
    output reg valid_out
);

reg signed [10:0] gx, gy;
reg [11:0] mag;

always @(posedge clk) begin
    if (rst) begin
        pixel_out <= 0;
        valid_out <= 0;
    end
    else if (valid_in) begin
        gx = (p6 + 2*p7 + p8) - (p0 + 2*p1 + p2);
        gy = (p0 + 2*p3 + p6) - (p2 + 2*p5 + p8);

        mag = (gx[10] ? -gx : gx) + (gy[10] ? -gy : gy);

        pixel_out <= (mag > 255) ? 8'hFF : mag[7:0];
        valid_out <= 1;
    end
    else valid_out <= 0;
end

endmodule
```

11.4 VGA Controller

```
module vga_controller(
    input clk,
    input rst,
    output reg hsync,
    output reg vsync,
    output reg video_on,
    output reg [9:0] x,
    output reg [9:0] y,
    output p_tick
);
```

```

parameter H_DISPLAY=640, H_FRONT=16, H_SYNC=96, H_BACK=48, H_TOTAL=800;
parameter V_DISPLAY=480, V_FRONT=10, V_SYNC=2, V_BACK=33, V_TOTAL=525;

reg [1:0] div;
assign p_tick = (div == 0);

always @(posedge clk) begin
    if (rst) div <= 0;
    else div <= div + 1;
end

always @(posedge clk) begin
    if (rst) begin x<=0; y<=0; end
    else if (p_tick) begin
        if (x == H_TOTAL-1) begin
            x <= 0;
            if (y == V_TOTAL-1) y <= 0;
            else y <= y + 1;
        end else x <= x + 1;
    end
end

always @(posedge clk) begin
    hsync <= ~((x >= H_DISPLAY+H_FRONT) && (x < H_DISPLAY+H_FRONT+H_SYNC));
    vsync <= ~((y >= V_DISPLAY+V_FRONT) && (y < V_DISPLAY+V_FRONT+V_SYNC));
    video_on <= (x < H_DISPLAY) && (y < V_DISPLAY);
end

endmodule

```

11.5 Python Code

```

import cv2
import numpy as np

INPUT_IMAGE = INPUT_IMAGE = r"C:\Users\utkarsh\Desktop\UT\DSD_Project\Hardware-Implementati
OUTPUT_COE = "fpga3_image.coe"
SIZE = (256, 256) # FPGA-friendly size

img = cv2.imread(INPUT_IMAGE)

if img is None:
    print("Error: Image not found!")
    exit()

```

```

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

gray_resized = cv2.resize(gray, SIZE)

cv2.imshow("Grayscale Image", gray_resized)
cv2.waitKey(0)
cv2.destroyAllWindows()

with open(OUTPUT_COE, "w") as f:
    f.write("memory_initialization_radix=10;\n")
    f.write("memory_initialization_vector=\n")

    flat_pixels = gray_resized.flatten()

    for i, pixel in enumerate(flat_pixels):
        if i == len(flat_pixels) - 1:
            f.write(f"{int(pixel)};")
        else:
            f.write(f"{int(pixel)},\n")

print("COE file generated successfully!")

```

11.6 Constraint File

```

## Clock signal
set_property -dict { PACKAGE_PIN E3 IOSTANDARD LVCMOS33 } [get_ports { clk }];
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { clk }];

set_property -dict { PACKAGE_PIN N17 IOSTANDARD LVCMOS33 } [get_ports { rst }];

## VGA Connector

set_property -dict { PACKAGE_PIN A3 IOSTANDARD LVCMOS33 } [get_ports { vga_r[0] }]
set_property -dict { PACKAGE_PIN B4 IOSTANDARD LVCMOS33 } [get_ports { vga_r[1] }]
set_property -dict { PACKAGE_PIN C5 IOSTANDARD LVCMOS33 } [get_ports { vga_r[2] }]
set_property -dict { PACKAGE_PIN A4 IOSTANDARD LVCMOS33 } [get_ports { vga_r[3] }]

set_property -dict { PACKAGE_PIN C6 IOSTANDARD LVCMOS33 } [get_ports { vga_g[0] }]
set_property -dict { PACKAGE_PIN A5 IOSTANDARD LVCMOS33 } [get_ports { vga_g[1] }]
set_property -dict { PACKAGE_PIN B6 IOSTANDARD LVCMOS33 } [get_ports { vga_g[2] }]
set_property -dict { PACKAGE_PIN A6 IOSTANDARD LVCMOS33 } [get_ports { vga_g[3] }]

set_property -dict { PACKAGE_PIN B7 IOSTANDARD LVCMOS33 } [get_ports { vga_b[0] }]
set_property -dict { PACKAGE_PIN C7 IOSTANDARD LVCMOS33 } [get_ports { vga_b[1] }]

```

```
set_property -dict { PACKAGE_PIN D7 IOSTANDARD LVCMOS33 } [get_ports { vga_b[2] }]
set_property -dict { PACKAGE_PIN D8 IOSTANDARD LVCMOS33 } [get_ports { vga_b[3] }]

set_property -dict { PACKAGE_PIN B11 IOSTANDARD LVCMOS33 } [get_ports { hsync }]
set_property -dict { PACKAGE_PIN B12 IOSTANDARD LVCMOS33 } [get_ports { vsync }]
```